

Java Cheatsheet

Unit 1 - Core Java

Location: `$JAVA_HOME/lib/modules` (module `java.base`) - auto-available, no extra JAR needed.

```
import java.lang.*; // auto-imported: String, Thread, Exception, System
import java.util.*; // Arrays, Scanner, ArrayList, HashMap
import java.io.*;   // File, InputStream, OutputStream, Reader, Writer, RandomAccessFile
```

Java Architecture: Source (`.java`) → `javac` compiler → Bytecode (`.class`) → JVM interprets at runtime.

Buzzwords: Simple, OOP, Robust, Secure, Portable, Architecture-neutral, Interpreted, High-performance, Multithreaded, Dynamic.

Path vs ClassPath: `PATH` points to `java` / `javac` binaries. `CLASSPATH` points to `.class` files / JARs. Set: `export CLASSPATH=.:lib/*` or `java -cp .:lib/* MyApp`.

Sample Program:

```
// Save as Hello.java → javac Hello.java → java Hello
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

You want to...	Use
Store text	<code>String</code> , <code>StringBuilder</code>
Store a list/map	<code>ArrayList</code> , <code>HashMap</code>
Store fixed-size values	<code>int[] arr = new int[5];</code> or <code>int[] arr = {1,2,3};</code>
Loop over array/ collection	<code>for (int x : arr)</code> (for-each / enhanced for)
Define a class	<code>class Car { String color; void drive() {} }</code> → <code>new Car()</code>
Overload a method	<code>void m(int x) {}</code> + <code>void m(String s) {}</code> (same name, different params)
Override a method	<code>@Override void m() {}</code> in subclass (same signature, different body)
Access privileges	<code>private</code> (class), <code>default</code> (package), <code>protected</code> (package+subclasses), <code>public</code> (everywhere)
Inheritance	<code>class B extends A {}</code> (single inheritance only)
Run code in parallel	<code>Thread</code> , <code>Runnable</code>
Handle errors	<code>try {} catch(Exception e) {} finally {}</code>
Read/write a file	<code>FileReader</code> / <code>FileWriter</code> or <code>InputStream</code> / <code>OutputStream</code>
Random access file	<code>RandomAccessFile("file","rw")</code> → <code>seek(pos)</code> , <code>readInt()</code> , <code>writeUTF()</code>
Read/write objects	<code>ObjectInputStream</code> / <code>ObjectOutputStream</code> + <code>implements Serializable</code>
Throw an exception	<code>throw new Exception("msg")</code>
Declare an exception	<code>void m() throws IOException {}</code>

Code snippets:

```

// Interface
interface A { void m(); }
class B implements A { public void m() {} }

// Inner class
class Outer { class Inner {} } // Outer.Inner i = new Outer().new Inner();

// Static nested class
class Outer { static class Nested {} } // new Outer.Nested();

// Final - subclass can't override
final void m() {}

// Static - call as Class.m() (no object)
static void m() {}

// Custom exception + throw / throws
class MyException extends Exception { MyException(String s) { super(s); } }
void validate(int age) throws MyException {
    if (age < 18) throw new MyException("Too young");
}

// Thread states: NEW → RUNNABLE → BLOCKED/WAITING → TERMINATED
Thread t = new Thread(() -> {}); // NEW
t.start(); // RUNNABLE
Thread.sleep(1000); // TIMED_WAITING
synchronized(lock) { lock.wait(); } // WAITING → BLOCKED

// Thread properties
t.setName("worker"); t.setDaemon(true); t.setPriority(Thread.NORM_PRIORITY);

// Thread (extend)
class T extends Thread { public void run() {} } // new T().start();

// Thread (Runnable)
class T implements Runnable { public void run() {} } // new Thread(new T()).start();

// Synchronized - one thread at a time
synchronized void m() {}

// Thread priority: 1 (MIN) to 10 (MAX)
t.setPriority(Thread.MAX_PRIORITY);

// Byte streams
FileInputStream fis = new FileInputStream("f.dat"); int b = fis.read(); fis.close();
FileOutputStream fos = new FileOutputStream("f.dat"); fos.write(65); fos.close();

// Character streams (text)
FileReader fr = new FileReader("f.txt"); int ch = fr.read(); fr.close();
FileWriter fw = new FileWriter("f.txt"); fw.write("Hello"); fw.close();
BufferedReader br = new BufferedReader(new FileReader("f.txt")); br.readLine(); br.close();

// RandomAccessFile
RandomAccessFile raf = new RandomAccessFile("f.dat", "rw");
raf.seek(10); int d = raf.readInt(); raf.writeUTF("hello"); raf.close();

// Object serialization
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("obj.dat"));
oos.writeObject(myObj); oos.close();
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("obj.dat"));
MyClass obj = (MyClass) ois.readObject(); ois.close();

```

Unit 2 - Swing

Location: module `java.desktop` in JDK - no extra JAR needed.

```
import javax.swing.*; // JFrame, JButton, JLabel, JTextField, JTable, JTree, JMenu
import java.awt.*;    // FlowLayout, BorderLayout, GridLayout, Color, Font
import java.applet.*; // Applet, AppletStub
```

AWT vs Swing: AWT uses native OS widgets (heavyweight), platform-dependent. Swing is pure Java (lightweight), pluggable Look & Feel, richer components.

Swing Class Hierarchy: `Component` → `Container` → `JComponent` → `JPanel`, `JLabel`, `AbstractButton` → `JButton`, `JToggleButton` → `JCheckBox` / `JRadioButton`, `JTextComponent` → `JTextField` / `TextArea`, etc.

Component vs Container: `Component` is any GUI element (button, label). `Container` is a component that holds other components (`JPanel`, `JFrame`). Use `add(child)` to nest.

Applet Lifecycle: `init()` (load) → `start()` (visible) → `stop()` (hidden) → `destroy()` (unload). `paint(Graphics g)` called as needed.

Layouts:

Layout	Behavior
No layout	<code>setLayout(null)</code> + <code>setBounds(x,y,w,h)</code> — absolute positioning
<code>FlowLayout</code>	Left-to-right row, wraps
<code>BorderLayout</code>	NORTH, SOUTH, EAST, WEST, CENTER (default for <code>JFrame</code>)
<code>GridLayout</code>	Equal-sized grid, <code>new GridLayout(rows, cols)</code>
<code>GridBagLayout</code>	Flexible grid, each cell sized via <code>GridBagConstraints</code>
<code>GroupLayout</code>	Horizontal + vertical sequential/parallel groups (NetBeans default)

You want to...	Use
A window	<code>JFrame</code>
A popup/custom dialog	<code>JDialog</code>
Arrange buttons in a row	<code>JPanel</code> + <code>FlowLayout</code>
Arrange in a grid	<code>GridLayout</code> or <code>GridBagLayout</code> or <code>GroupLayout</code>
A text box	<code>JTextField</code>
A password box	<code>JPasswordField</code>
A text area	<code>TextArea</code> + <code>JScrollPane</code>
A checkbox	<code>JCheckBox</code>
Radio buttons	<code>JRadioButton</code> + <code>ButtonGroup</code>
A dropdown	<code>JComboBox</code>
A slider	<code>JSlider</code>
A menu bar	<code>JMenuBar</code> → <code>JMenu</code> → <code>JMenuItem</code>
Icon in menu item	<code>JMenuItem("Cut", new ImageIcon("cut.png"))</code> or <code>setIcon(icon)</code>
Checkbox/radio in menu	<code>JCheckBoxMenuItem</code> , <code>JRadioButtonMenuItem</code>
Popup menu on right-click	<code>JPopupMenu</code> → <code>setComponentPopupMenu(popup)</code>
Enable/disable item	

You want to...	Use
	<code>item.setEnabled(false)</code>
A table	<code>JTable</code> + <code>DefaultTableModel</code>
A tree	<code>JTree</code> + <code>DefaultMutableTreeNode</code>
A file picker	<code>JFileChooser</code>
A tooltip	<code>comp.setTooltipText("help text")</code>
A message popup	<code>JOptionPane.showMessageDialog()</code>
A color picker	<code>JColorChooser.showDialog()</code>
Toolbar	<code>JToolBar</code> + <code>JButton</code> + <code>ImageIcon</code>

Code snippets:

```
// Window
JFrame f = new JFrame(); f.setSize(400,300); f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

// BorderLayout (default for JFrame)
f.add(new JButton("North"), BorderLayout.NORTH);
f.add(new JButton("Center"), BorderLayout.CENTER);

// No layout / absolute positioning
f.setLayout(null);
JButton b = new JButton("Go"); b.setBounds(50, 50, 100, 30); f.add(b);

// GroupLayout
GroupLayout layout = new GroupLayout(panel);
panel.setLayout(layout);
layout.setHorizontalGroup(layout.createSequentialGroup()
    .addComponent(label).addComponent(textField));
layout.setVerticalGroup(layout.createParallelGroup()
    .addComponent(label).addComponent(textField));

// GridBag
GridBagConstraints g = new GridBagConstraints();
g.gridx = 0; g.gridy = 0; panel.add(comp, g);

// Menu + Mnemonic
JMenu m = new JMenu("File"); m.setMnemonic('F');

// Accelerator (Ctrl+S)
JMenuItem i = new JMenuItem("Save");
i.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_S, KeyEvent.CTRL_MASK));

// Icon in menu item
JMenuItem cut = new JMenuItem("Cut", new ImageIcon("cut.png"));
cut.setDisabledIcon(new ImageIcon("cut_gray.png")); cut.setEnabled(false);

// Checkbox & radio in menu
JCheckBoxMenuItem cb = new JCheckBoxMenuItem("Bold");
JRadioButtonMenuItem rb = new JRadioButtonMenuItem("Left");

// Popup menu
JPopupMenu popup = new JPopupMenu();
popup.add(new JMenuItem("Copy")); popup.add(new JMenuItem("Paste"));
component.setComponentPopupMenu(popup);

// Internal frame
JDesktopPane dp = new JDesktopPane();
JInternalFrame ifr = new JInternalFrame(); ifr.setVisible(true);
dp.add(ifr); f.add(dp);

// Tooltip
b.setTooltipText("Click to submit");

// Applet -> <applet code="A.class" width=200 height=100>
class A extends Applet {
    public void init() {}
}
```

```
public void start() {}  
public void paint(Graphics g) { g.drawString("Hi",20,20); }  
public void stop() {}  
public void destroy() {}  
}
```

Unit 3 - Event Handling

Location: same module as Swing (`java.desktop`).

```
import java.awt.event.*; // ActionListener, KeyListener, MouseListener, WindowListener
import javax.swing.event.*; // ListSelectionListener, ChangeListener, DocumentListener
```

When user does...	Use listener
Clicks a button	<code>ActionListener</code> (use <code>e.getActionCommand()</code> to identify button)
Types a key	<code>KeyListener</code> (<code>keyPressed</code> , <code>keyReleased</code> , <code>keyTyped</code>)
Focuses a component	<code>FocusListener</code> (<code>focusGained</code> , <code>focusLost</code>)
Clicks mouse	<code>MouseListener</code>
Moves mouse	<code>MouseMotionListener</code>
Closes window	<code>WindowListener</code>
Selects a checkbox	<code>ItemListener</code>
Moves a slider	<code>ChangeListener</code>
Selects a table row	<code>ListSelectionListener</code>

Code snippet:

```
// ActionEvent (button click) – use getActionCommand() or getSource()
class Foo implements ActionListener {
    JButton b = new JButton("Go");
    Foo() { b.setActionCommand("go"); b.addActionListener(this); }
    public void actionPerformed(ActionEvent e) {
        String cmd = e.getActionCommand(); // "go"
        if (e.getSource() == b) { /* do stuff */ }
    }
}

// FocusEvent
class Foo extends FocusAdapter {
    public void focusGained(FocusEvent e) { comp.setBackground(Color.YELLOW); }
    public void focusLost(FocusEvent e) { comp.setBackground(Color.WHITE); }
}

// MouseEvent (use MouseAdapter to skip unused methods)
class Foo extends MouseAdapter {
    public void mouseClicked(MouseEvent e) {}
}

// KeyEvent
class Foo extends KeyAdapter {
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_ENTER) { /* submit */ }
    }
}
```

Unit 4 - JDBC

Location: `java.sql` module in JDK + your DB driver JAR (e.g., `WEB-INF/lib/h2-2.2.224.jar`).

```
import java.sql.*; // DriverManager, Connection, Statement, PreparedStatement, ResultSet, SQLException
```

You want to...	Use
Connect to DB	<code>DriverManager.getConnection(url)</code>
Run a plain query	<code>Statement.executeQuery("SELECT ...")</code>
Parameterized query	<code>PreparedStatement</code> + <code>setString()</code> (prevents SQL injection)
Loop through results	<code>while (rs.next()) { rs.getString("col"); }</code>
Insert/update/delete	<code>executeUpdate()</code>
Scrollable result set	<code>ResultSet.TYPE_SCROLL_SENSITIVE</code>
Updateable result set	<code>ResultSet.CONCUR_UPDATABLE</code> + <code>updateRow()</code>
Work offline	<code>CachedRowSet</code>
Multiple results from stored proc	<code>st.execute(sql) → do { rs = st.getResultSet(); } while (st.getMoreResults());</code>
Transactions	<code>setAutoCommit(false) → commit/rollback</code>

Code snippet:

```
Class.forName("org.h2.Driver"); // or com.mysql.jdbc.Driver
Connection con = DriverManager.getConnection("jdbc:h2:./test");

// DDL
con.createStatement().executeUpdate("CREATE TABLE users (id INT, name VARCHAR(50))");

// INSERT (PreparedStatement)
PreparedStatement ps = con.prepareStatement("INSERT INTO users VALUES (?,?)");
ps.setInt(1, 1); ps.setString(2, "Alice"); ps.executeUpdate();

// SELECT
ResultSet rs = con.createStatement().executeQuery("SELECT * FROM users");
while (rs.next()) { rs.getInt("id"); rs.getString("name"); }

// UPDATE / DELETE
con.createStatement().executeUpdate("UPDATE users SET name='Bob' WHERE id=1");

// Multiple results (stored procedure returns multiple ResultSets)
boolean hasResults = st.execute("CALL myProc()");
do {
    if (hasResults) { ResultSet r = st.getResultSet(); /* process */ }
} while (st.getMoreResults());

// Scrollable
Statement st = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY);

// Transaction
con.setAutoCommit(false);
try { /* stmts */ con.commit(); } catch (Exception e) { con.rollback(); }
con.setAutoCommit(true);

// SQL escapes
"SELECT * FROM users WHERE name LIKE '100%' {escape '0'}"
"SELECT {fn UCASE(name)} FROM users"
```

Unit 5 - Networking

Location: `java.base` module in JDK for `java.net.*`; `javax.mail.jar` (external) for email.

```
import java.net.*; // Socket, ServerSocket, DatagramSocket, URL, URLConnection
// For email add javax.mail.jar: import javax.mail.*; import javax.mail.internet.*;
```

You want to...	Use
TCP client	<code>Socket s = new Socket("host", port);</code> → <code>getInputStream()</code> / <code>getOutputStream()</code>
TCP server	<code>ServerSocket ss = new ServerSocket(port);</code> → <code>Socket s = ss.accept();</code>
UDP send	<code>DatagramSocket ds = new DatagramSocket();</code> → <code>ds.send(new DatagramPacket(data, len, addr, port));</code>
UDP receive	<code>DatagramSocket ds = new DatagramSocket(port);</code> → <code>ds.receive(new DatagramPacket(buf, len));</code>
Download webpage	<code>URL("https://...").openStream()</code>
Send email	<code>Session</code> → <code>MimeMessage</code> → <code>Transport.send()</code>
Receive email	<code>Session</code> → <code>Store</code> → <code>Folder</code> → <code>Message[]</code>

Code snippets:

TCP:

```
// Client
Socket s = new Socket("localhost", 4241);
PrintWriter out = new PrintWriter(s.getOutputStream(), true);
Scanner in = new Scanner(s.getInputStream());
out.println("Hello"); System.out.println(in.nextLine());
s.close();

// Server
ServerSocket ss = new ServerSocket(4241);
Socket s = ss.accept();
Scanner in = new Scanner(s.getInputStream());
PrintWriter out = new PrintWriter(s.getOutputStream(), true);
System.out.println(in.nextLine()); out.println("Hi");
s.close(); ss.close();
```

UDP:

```
// Send
DatagramSocket ds = new DatagramSocket();
byte[] data = "Hello".getBytes();
ds.send(new DatagramPacket(data, data.length, InetAddress.getByName("localhost"), 9876));

// Receive
DatagramSocket ds = new DatagramSocket(9876);
byte[] buf = new byte[1024];
DatagramPacket p = new DatagramPacket(buf, buf.length);
ds.receive(p);
System.out.println(new String(p.getData(), 0, p.getLength()));
```

Email (send):

```
Properties p = new Properties(); p.put("mail.smtp.host", "smtp.gmail.com");
Session s = Session.getInstance(p);
MimeMessage m = new MimeMessage(s);
m.setFrom(new InternetAddress("from@x.com"));
m.addRecipient(Message.RecipientType.TO, new InternetAddress("to@x.com"));
```



```
m.setSubject("Test"); m.setText("Hello");  
Transport.send(m);
```

Email (receive):

```
Properties p = new Properties(); p.put("mail.store.protocol", "imaps");  
Session s = Session.getInstance(p);  
Store store = s.getStore("imaps"); store.connect("imap.gmail.com", "user@x.com", "pass");  
Folder inbox = store.getFolder("INBOX"); inbox.open(Folder.READ_ONLY);  
Message[] msgs = inbox.getMessages();  
for (Message msg : msgs) {  
    System.out.println(msg.getFrom()[0] + ": " + msg.getSubject());  
}  
inbox.close(false); store.close();
```

Unit 6 - JavaFX

Location: External JavaFX SDK (not in JDK by default). Download `javafx-controls.jar` etc.

```
import javafx.application.*; // Application, Platform
import javafx.stage.*;      // Stage, Scene, FileChooser
import javafx.scene.*;      // Node, Group, Image
import javafx.scene.layout.*; // FlowPane, BorderPane, HBox, VBox, GridPane
import javafx.scene.control.*; // Label, Button, TextField, MenuBar, TableView
import javafx.event.*;      // Event, ActionEvent, EventHandler
```

You want to...	Use
Start a JavaFX app	<code>extends Application + @Override start()</code>
A window	<code>Stage</code> (passed to <code>start()</code>)
A scene	<code>Scene(rootNode, width, height)</code>
Arrange horizontally	<code>HBox</code>
Arrange vertically	<code>VBox</code>
Flexible grid	<code>GridPane</code>
A button	<code>Button + setOnAction(e -> {})</code>
A checkbox	<code>CheckBox + setOnAction(e -> box.isSelected())</code>
A radio button	<code>RadioButton + ToggleGroup</code>
A hyperlink	<code>Hyperlink + setOnAction(e -> getHostServices().showDocument(uri))</code>
A menu bar	<code>MenuBar → Menu → MenuItem</code>
A tooltip	<code>comp.setTooltip(new Tooltip("text"))</code>
A file chooser	<code>FileChooser → showOpenDialog(stage)</code>
An alert	<code>Alert(AlertType.INFORMATION)</code>

Code snippet:

```
public class Main extends Application {
    public void start(Stage stage) {
        Button b = new Button("Click");
        b.setTooltip(new Tooltip("Click this button"));
        b.setOnAction(e -> System.out.println("Hi"));

        CheckBox cb = new CheckBox("Agree");

        ToggleGroup tg = new ToggleGroup();
        RadioButton r1 = new RadioButton("Male"); r1.setToggleGroup(tg);
        RadioButton r2 = new RadioButton("Female"); r2.setToggleGroup(tg);

        Hyperlink link = new Hyperlink("Visit");
        link.setOnAction(e -> getHostServices().showDocument("https://example.com"));

        MenuBar menuBar = new MenuBar();
        Menu fileMenu = new Menu("File");
        MenuItem openItem = new MenuItem("Open");
        fileMenu.getItems().add(openItem);
        menuBar.getMenus().add(fileMenu);

        VBox root = new VBox(10, menuBar, b, cb, r1, r2, link);
        stage.setScene(new Scene(root, 300, 250));
        stage.show();
    }
}
```

```
public static void main(String[] args) { launch(args); }  
}
```

Unit 7 - Servlets & JSP

Location: `$TOMCAT_HOME/lib/servlet-api.jar` + `jsp-api.jar` + `el-api.jar` . Add to classpath when compiling.

```
import javax.servlet.*;           // Servlet, ServletConfig, ServletContext, RequestDispatcher
import javax.servlet.http.*;      // HttpServlet, HttpServletRequest, HttpServletResponse, HttpSession, Cookie
import javax.servlet.annotation.*; // WebServlet
```

JSP implicit objects: `request` , `response` , `session` , `application` , `out` , `config` , `page` , `pageContext`

JSP scopes (narrow → wide): `page` → `request` → `session` → `application`

Web Container (Servlet Container): Part of Tomcat that manages servlet lifecycle, maps URLs to servlets, handles HTTP requests/responses, manages session, security, and multithreading.

You want to...	Use
Handle form POST	<code>HttpServlet + doPost() → request.getParameter("name")</code>
Handle GET	<code>doGet() → response.getWriter().println()</code>
Store user data	<code>session.setAttribute("key", value)</code>
Remember across visits	<code>Cookie c = new Cookie("k","v"); response.addCookie(c);</code>
Read cookies	<code>request.getCookies()</code>
Redirect	<code>response.sendRedirect("page.jsp")</code>
Write Java in JSP	<code><% ... %></code> scriptlet
Output value in JSP	<code><%= expr %></code> or <code>\${expr}</code>
Define method in JSP	<code><%! ... %></code> declaration
Comment in JSP	<code><!-- ... --></code> (hidden from client, unlike <code><!-- --></code>)
Import in JSP	<code><%@ page import="java.util.*" %></code>
Map URL to servlet	<code>@WebServlet("/path")</code>

Code snippets:

Servlet:

```
@WebServlet("/FormServlet")
public class FormServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println(req.getParameter("user"));
    }
}
```

JSP:

```
<%@ page language="java" contentType="text/html" %>
<html><body>
    <!-- This is a JSP comment (hidden from client) -->
    <%
        String name = request.getParameter("fname");
        out.println(name);
    %>
    <p>${param.fname}</p>
</body></html>
```

Cookie:

```
// Set
Cookie c = new Cookie("user", "Alice");
c.setMaxAge(86400); // 1 day
response.addCookie(c);

// Read
Cookie[] cs = request.getCookies();
for (Cookie c : cs) { c.getName(); c.getValue(); }
```

Session:

```
HttpSession s = request.getSession();
s.setAttribute("key", "val"); // store
String v = (String) s.getAttribute("key"); // read
s.invalidate(); // destroy (logout)
```

Java Web Frameworks: Frameworks built on top of Servlets/JSP that simplify web development. - **Spring MVC:**

Annotations (`@Controller` , `@RequestMapping`), dependency injection, REST APIs. - **Struts 2:** Action-based, XML/annotation config, interceptors. - **JSF (JavaServer Faces):** Component-based, managed beans, Facelets (XHTML views).

- **JAX-RS (REST):** Annotations like `@GET` , `@Path("/api")` for RESTful web services.

Unit 8 - RMI & CORBA

Location: `java.rmi` module in JDK for RMI. CORBA (`org.omg.CORBA.*`) removed from JDK 11+.

```
import java.rmi.*;           // Remote, RemoteException
import java.rmi.server.*;    // UnicastRemoteObject
import java.rmi.registry.*;  // Registry, LocateRegistry
import java.rmi.Naming;      // rebind(), lookup()
// CORBA (only on older JDK): import org.omg.CORBA.*; import org.omg.CosNaming.*;
```

You want to...	Use
Call method on another machine	RMI
Define remote service	<code>extends UnicastRemoteObject</code> , implement a <code>Remote</code> interface
Start registry	<code>LocateRegistry.createRegistry(1099)</code>
Register service	<code>Naming.rebind("rmi://localhost/MyService", obj)</code>
Call from client	<code>Naming.lookup("rmi://localhost/MyService")</code>
Cross-language calls	CORBA + IDL file

RMI Architecture: Client → Stub → Network → Skeleton → Remote Object. Registry (`rmiregistry`) holds service references at port 1099.

CORBA Architecture: Client → Stub (IDL-generated) → ORB → IIOP → ORB → Skeleton (IDL-generated) → Servant. Supports multiple languages via IDL.

RMI code:

```
// 1. Remote interface
import java.rmi.*;
public interface Adder extends Remote {
    int add(int x, int y) throws RemoteException;
}

// 2. Implementation
import java.rmi.server.*;
public class AdderImpl extends UnicastRemoteObject implements Adder {
    public AdderImpl() throws RemoteException {}
    public int add(int x, int y) { return x + y; }
}

// 3. Server
AdderImpl obj = new AdderImpl();
LocateRegistry.createRegistry(1099);
Naming.rebind("rmi://localhost/Add", obj);

// 4. Client
Adder a = (Adder) Naming.lookup("rmi://localhost/Add");
System.out.println(a.add(3, 4)); // 7
```

CORBA:

```
// HelloApp.idl
module HelloApp {
    interface Hello {
        string sayHello();
    };
};

// Compile: idlj -fall HelloApp.idl → generates _HelloStub.java, _HelloImplBase.java, etc.

// -- Server --
import HelloApp.*;
import org.omg.CORBA.*;
import org.omg.CosNaming.*;
```

```

public class HelloServer {
    public static void main(String[] args) throws Exception {
        ORB orb = ORB.init(args, null);
        HelloImpl impl = new HelloImpl();
        orb.connect(impl);

        org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
        NamingContextExt nc = NamingContextExtHelper.narrow(objRef);
        nc.rebind(nc.to_name("Hello"), impl);

        java.lang.Object sync = new java.lang.Object();
        synchronized (sync) { sync.wait(); }
    }
}

// -- Servant --
import HelloApp.*;
import org.omg.CORBA.*;

class HelloImpl extends HelloPOA {
    public String sayHello() { return "Hello from CORBA!"; }
}

// -- Client --
import HelloApp.*;
import org.omg.CORBA.*;
import org.omg.CosNaming.*;

public class HelloClient {
    public static void main(String[] args) throws Exception {
        ORB orb = ORB.init(args, null);
        org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
        NamingContextExt nc = NamingContextExtHelper.narrow(objRef);
        Hello h = HelloHelper.narrow(nc.resolve(nc.to_name("Hello")));
        System.out.println(h.sayHello());
    }
}

```